

同步方法之四



锁的获取与放弃

北京理工大学计算机学院
金旭亮

锁重入



可以在一个线程中嵌套（或递归地）调用同步方法。



比如，a和b方法都是同步方法，某线程获得了锁独占性地执行a方法时，如果a方法内部又调用了b方法，这种调用是可以顺畅进行的，这实际上就是同一个线程申请了同一把锁两次。



已经拥有了锁，再次申请锁，此即“**锁重入**”。

锁重入示例

```
public class ReentrantLockDemo {  
    1 usage  
    public synchronized void a() {  
        -----  
        var threadName :String = Thread.currentThread().getName();  
        System.out.println(threadName + "调用同步方法a");  
        //继续调用另一个同步方法  
        b();  
    }  
  
    1 usage  
    public synchronized void b() {  
        var threadName :String = Thread.currentThread().getName();  
        System.out.println(threadName + "调用同步方法b");  
    }  
  
    public static void main(String[] args) {  
        new Thread(() -> {  
            new ReentrantLockDemo().a();  
        }).start();  
    }  
}
```

Run: ReentrantLockDemo x

"C:\Program Files\Java\jdk-17\bin\java.exe"

Thread-0调用同步方法a

Thread-0调用同步方法b

Process finished with exit code 0

父子类型中的同步方法重写

```
class Parent {  
    4 usages  1 override  
    public synchronized void f() {  
        new ThreadHelper().process();  
    }  
}  
  
2 usages  
class Child extends Parent {  
    4 usages  
    @Override  
    public void f() {  
        new ThreadHelper().process();  
    }  
}
```

父类中定义了一个同步方法，子类重写了，但没有加上synchronized关键字.....

测试同步方法重写-1

//测试多线程调用父类的同步方法

1 usage

```
private static void testInvokeParent() {  
    Parent parent = new Parent();  
    new Thread(parent::f).start();  
    new Thread(parent::f).start();  
}
```



这个结果，应该没有什么奇怪的，两个线程访问同一个对象的同一个同步方法，肯定是排好队“一个一个来”的。

```
Run: ParentAndChildTest x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
线程Thread-0正在处理0  
线程Thread-0正在处理1  
线程Thread-0正在处理2  
线程Thread-0正在处理3  
线程Thread-1正在处理0  
线程Thread-1正在处理1  
线程Thread-1正在处理2  
线程Thread-1正在处理3  
Process finished with exit code 0
```

线程一

线程二

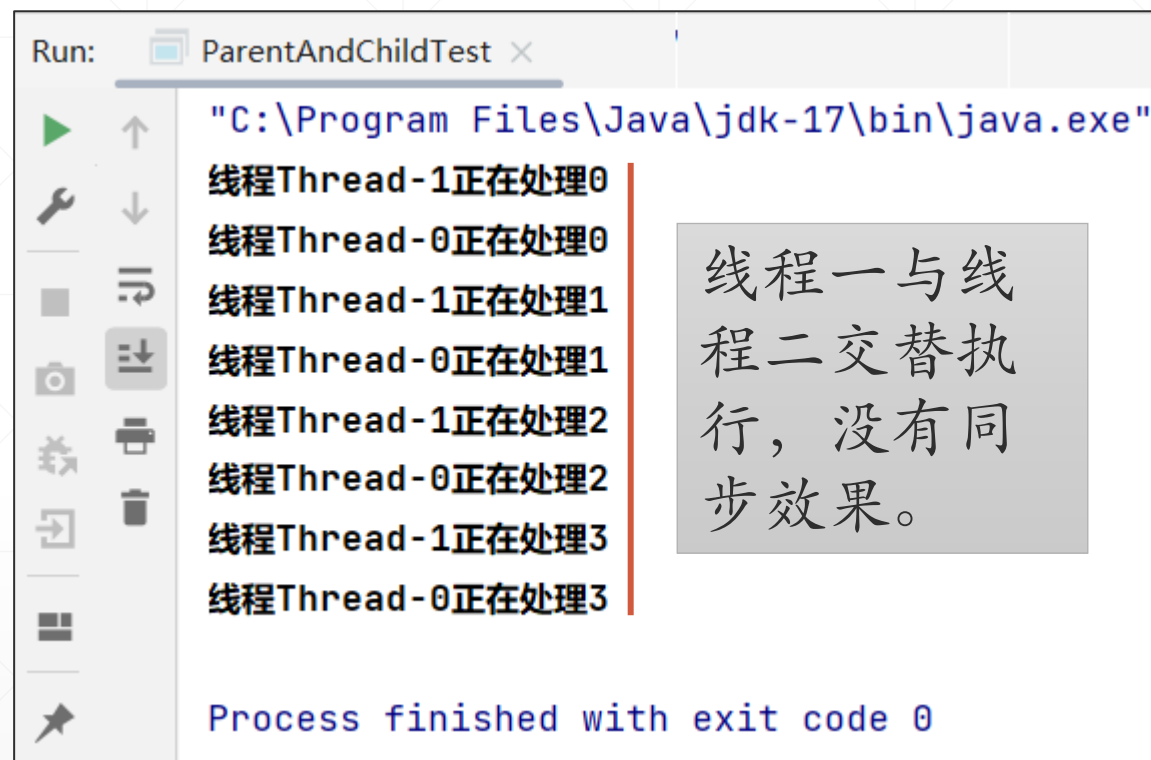
测试同步方法重写-2

//测试多线程调用子类的重写后的同步方法

1 usage

```
private static void testInvokeChild() {  
    Child child = new Child();  
    new Thread(child::f).start();  
    new Thread(child::f).start();  
}
```

子类重写了父类的同步方法，由于它没有加上synchronized关键字，就变成了普通方法。多线程调用时，是交替执行的，语句的顺序不可预知。



Run: ParentAndChildTest x

"C:\Program Files\Java\jdk-17\bin\java.exe"

线程Thread-1正在处理0
线程Thread-0正在处理0
线程Thread-1正在处理1
线程Thread-0正在处理1
线程Thread-1正在处理2
线程Thread-0正在处理2
线程Thread-1正在处理3
线程Thread-0正在处理3

Process finished with exit code 0

线程一与线程二交替执行，没有同步效果。

小结



在继承条件下，如果父类中的某方法声明为同步方法，子类重写此方法时，也应该设置为同步方法，以保证线程安全。如果违反了这个规则，则多线程访问子对象的这个同步方法时，将不再是“线程安全”的了。

动手试验-1



如果将子类重写的方法改为以下这样：

```
class Child extends Parent {  
    //...  
    5 usages  
    @Override  
    public void f() {  
        super.f();  
    }  
}
```

多线程调用子类的f()方法，
结果会有不一样的吗？

动手试验-2



//测试多态场景

1 usage

```
private static void testPolymorphism() {  
    Parent parent = new Child();  
    new Thread(parent::f).start();  
    new Thread(parent::f).start();  
}
```

如果将代码改为左侧这样，应用了多态特性，程序的输出会是什么？

先想一想，然后上机验证，看看你的想法对不对。

锁争用带来的问题



同步方法中如果出现了死循环，将导致它所占用的锁迟迟不能释放，这样一搞，问题就大了.....

```
class MySyncClass {  
    //太自私了，占有锁就不再放手了~~~  
    1 usage  
    public synchronized void process1() {  
        var threadName :String = Thread.currentThread().getName();  
        //构建一个死循环  
        for (int i = 0; ; i++) {  
            System.out.println(threadName + "正在处理" + i);  
            try {  
                TimeUnit.SECONDS.sleep( timeout: 1);  
            } catch (InterruptedException e) {  
                throw new RuntimeException(e);  
            }  
        }  
    }  
    //我需要拥有锁才能运行  
    1 usage  
    public synchronized void process2() {  
        var threadName :String = Thread.currentThread().getName();  
        System.out.println(threadName + "正在调用process2()");  
    }  
}
```

锁争用测试



```
public class SyncLeadToStarve {  
    public static void main(String[] args)  
        throws InterruptedException {  
        var obj = new MySyncClass();  
        new Thread(obj::process1).start();  
        TimeUnit.SECONDS.sleep(timeout: 2);  
        new Thread(obj::process2).start();  
    }  
}
```



Run: SyncLeadToStarve x

"C:\Program Files\Java\jdk1...

Thread-0正在处理0
Thread-0正在处理1
Thread-0正在处理2
Thread-0正在处理3
Thread-0正在处理4
Thread-0正在处理5

由于第一个线程得到了锁，但一直不释放，结果，第二个线程被“饿死”了.....

如果线程函数中抛出了异常.....

```
public synchronized void process1() {  
    var threadName :String = Thread.currentThread().getName();  
    //构建一个死循环  
    for (int i = 0; ; i++) {  
        System.out.println(threadName + "正在处理" + i);  
        try {  
            TimeUnit.SECONDS.sleep(timeout: 1);  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
        if (i == 3) {  
            throw new RuntimeException("抛出异常");   
        }  
    }  
}
```

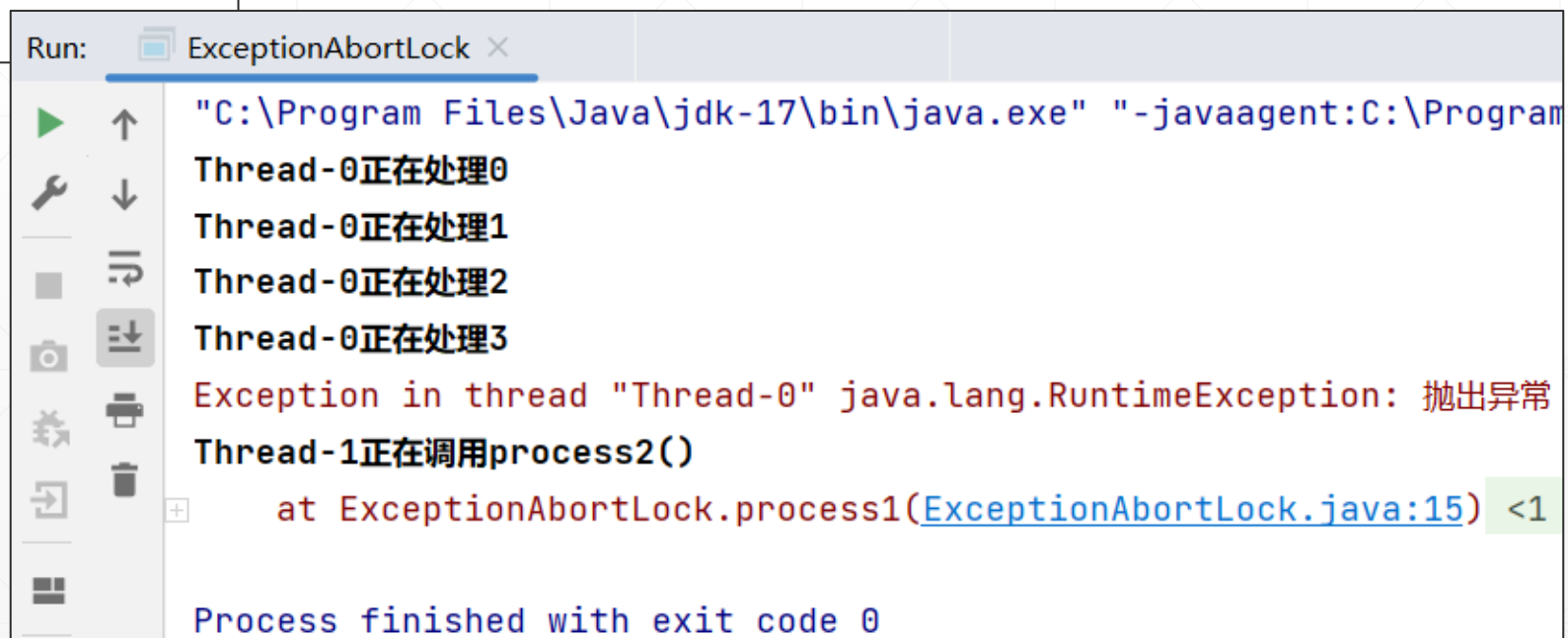
同步方法process1()执行时会抛出异常，另一个同步方法process2()则不会。

```
public synchronized void process2() {  
    var threadName :String = Thread.currentThread().getName();  
    System.out.println(threadName + "正在调用process2()");  
}
```

异常抛出会放弃锁

```
public static void main(String[] args)
    throws InterruptedException {
    var obj = new ExceptionAbortLock();
    new Thread(obj::process1).start();
    TimeUnit.SECONDS.sleep( timeout: 1);
    new Thread(obj::process2).start();
}
```

从运行结果上看，由于第一个线程的线程函数中抛出了异常，导致它所占有的锁被释放，现在，第二个线程就有机会得到锁并运行了.....



```
Run: ExceptionAbortLock x
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program
Thread-0正在处理0
Thread-0正在处理1
Thread-0正在处理2
Thread-0正在处理3
Exception in thread "Thread-0" java.lang.RuntimeException: 抛出异常
Thread-1正在调用process2()
    at ExceptionAbortLock.process1(ExceptionAbortLock.java:15) <1
Process finished with exit code 0
```